

Lesson 3: OS Structures and System Calls

Course: Operating Systems

Year Level: 3rd Year College (BSIT/BSCS)

Duration: 3 Hours

Learning Outcomes

Attitude

Appreciate the importance of structured and modular operating system design in developing reliable and efficient software.

Knowledge

- Explain the architecture of monolithic and microkernel operating systems.
- Describe the concept of a system call.
- Explain the transition between User Mode and Kernel Mode.

Skills

- Draw and compare monolithic and microkernel operating system architectures.
- Trace the execution flow of a system call such as `open()`.

What is an OS Structure?

- refers to the way an operating system is organized into different components or layers and how these components interact to manage computer hardware and software resources.

The main OS structures are:

Simple Structure

- The operating system is written as a single large program.
- All components can communicate directly.
- Easy to design but difficult to maintain.
- **Example:** MS-DOS.

The main OS structures are:.....

Layered Structure

- The OS is divided into layers.
- Each layer performs specific functions and communicates only with the adjacent layers.
- Easier to develop, debug, and maintain.
- Example:** THE Operating System.

Microkernel Structure

- Only essential services (such as process management and memory management) run in the kernel.
- Other services (file system, device drivers) run in user space.
- More secure and reliable but may have slightly lower performance.
- Example:** Minix, QNX.

Layered Operating System (OS) Structure

Operating System Structure

Applications

System Libraries

System Call Interface

Hardware

Each layer only communicates with the layer directly above or below it. This layered design makes the operating system **easier to develop, debug, maintain, and secure** because each layer has a specific responsibility.

User Applications

- Programs used by end users, such as web browsers, Microsoft Office, media players, and games.
-

System Programs

- Utilities and software that help users interact with the operating system, such as command-line shells, compilers, and file managers.

System Call Interface

- Acts as the communication bridge between user programs and the operating system kernel.
- Example system calls: `open()`, `read()`, `write()`, `fork()`, and `exit()`.

The main OS structures are:.....

Modular Structure

- The OS consists of separate modules that can be loaded or unloaded as needed.
- Provides flexibility and better performance.
- Example:** Linux.

Hybrid Structure

- Combines features of monolithic and microkernel designs.
- Balances performance and modularity.
- Example:** Windows, macOS.

Operating System Kernel

- The core of the operating system.
- Responsible for:

 - Process Management
 - Memory Management
 - File Management
 - Device Management
 - Security and Protection

Hardware

- The physical components of the computer, including the CPU, memory (RAM), storage devices, keyboard, mouse, monitor, printers, and network devices.

Major Operating System Structures

1. Monolithic Architecture

2. Microkernel Architecture

Monolithic Architecture

- ❑ A **Monolithic Architecture** is an operating system design where all OS services run together in a **single large kernel** operating in **kernel mode**. All components can directly communicate with each other because they share the same memory space.
- ❑ A **Monolithic Architecture** is an OS structure where all operating system services execute within one large kernel and directly interact with each other to manage system resources efficiently.

Applications (user-space)

App_1

App_2

App_3

User space
Kernel space

System Calls

OS

File System

I/O Interface

Network Interface

Memory manager

IPC

Process Scheduler

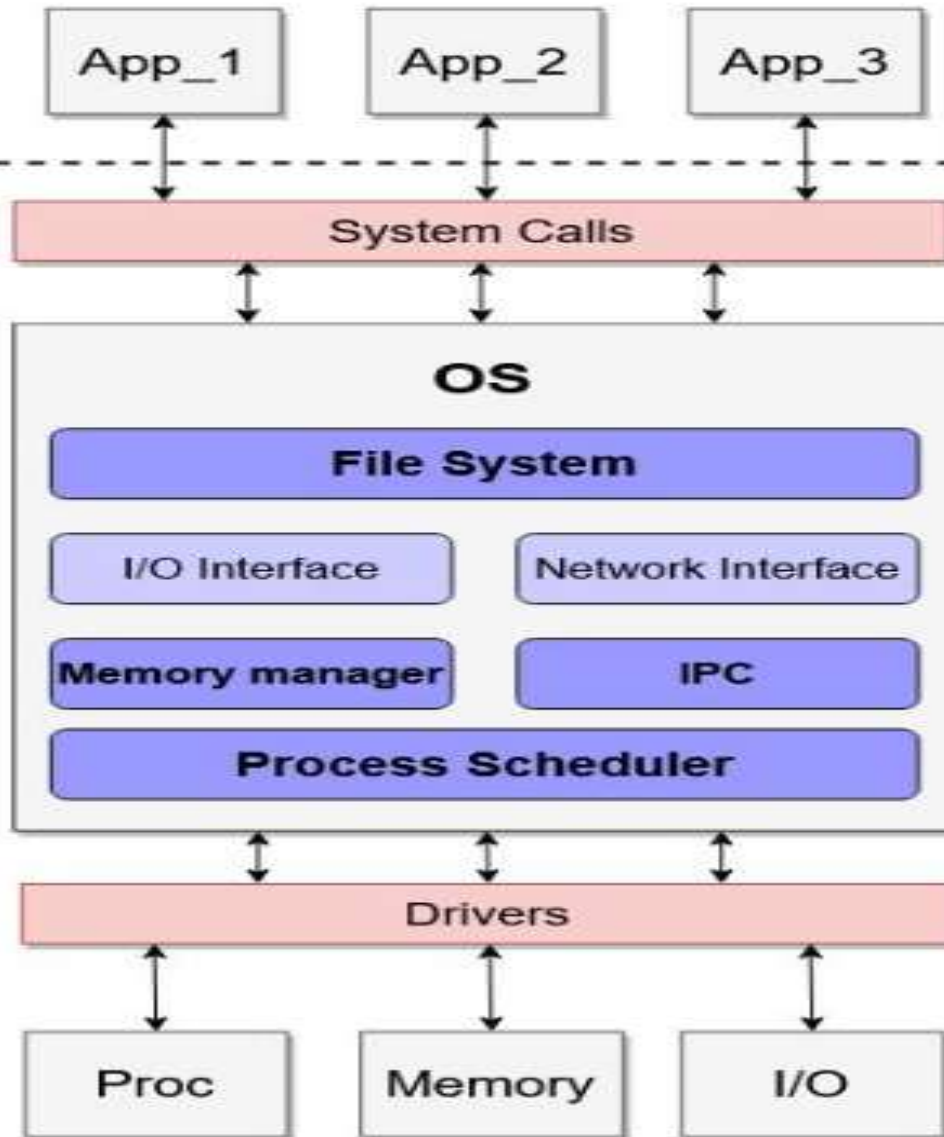
Drivers

Proc

Memory

I/O

Hardware



How It Works

- ❑ User applications request services through **system calls**.
- ❑ The request enters the **kernel**.
- ❑ Kernel components communicate directly with each other.
- ❑ The kernel accesses the hardware and returns the result to the application.

Characteristics

- ❑ **Single large kernel**
- ❑ All services run in **kernel space**
- ❑ Components communicate directly
- ❑ High performance due to fewer communication overheads
- ❑ Easier access to hardware resources

Advantages

- ✓ Fast execution and high performance
 - ✓ Efficient communication between components
 - ✓ Simple design for smaller systems
-

Disadvantages

- ✗ Large and complex kernel
- ✗ Difficult to maintain and debug
- ✗ An error in one component can crash the entire system
- ✗ Lower security because all services run with full privileges

Examples of Monolithic Operating Systems

- Linux (traditional monolithic kernel with modular support)
- UNIX
- MS-DOS

Comparison

Feature	Monolithic Kernel	Microkernel
Size	Large	Small
Performance	Faster	Slightly slower
Security	Lower	Higher
Reliability	Lower	Higher
Services	Inside kernel	Mostly outside kernel